CAP. 8 STRUCTURA PROGRAMELOR FORTRAN

8.1 Unități de program

Orice program, atunci când este executat de computer, îndeplineşte un scop, realizează a anumită misiune caracterizată de îndeplinirea unor obiective precise de prelucrare a datelor. Mulțimea obiectivelor impuse ce definesc o misiune de prelucrare a datelor se numeşte **task**. Proiectarea unui program începe întotdeauna cu descrierea taskului. Atunci când misiunea programului este prea complicată, se foloseşte tactica "divide et impera": un task T se împarte în subtaskuri T_1, T_2, …, T_n cu condiția ca îndeplinirea taskurilor T_1, T_2, …, T_n să însemne îndeplinirea taskului T.

Orice **program Fortran executabil** este alcătuit din una sau mai multe **unități de program**. Unitate de program poate fi: **programul principal** (main), un **subprogram extern** (subrutină sau funcție), un **modul**, sau un **bloc de date** (block data). Dacă programul este format dintr-o singură unitate de program, acea unitate, în mod obligatoriu, este programul principal. Dacă un program executabil îndeplineşte un task *T*, fiecare unitate de program corespunde unuia din subtaskurile T_1, T_2, …, T_n care contribuie la îndeplinirea taskului *T*.

Fiecare unitate de program este formată din instrucțiuni executabile şi neexecutabile (§8.2). Fiecare unitate de program începe cu o instrucțiune inițială specială şi se termină cu o instrucțiune END. Atunci când programul este alcătuit din mai multe unități de program, programul principal controlează şi ordinea în care se execută diferitele unități de program.

Subprogramul extern (funcție sau subrutină) se poate folosi pentru a efectua prelucrări asupra unor entități care sunt disponibile subprogramului extern. Aceste entități pot fi argumente ale subprogramului care se dau în referința de subprogram, sau, sunt făcute accesibile subprogramului prin alte mijloace, de exemplu, prin blocuri common. Pentru a apela o subrutină se foloseşte instrucțiunea CALL. Atunci când este nevoie de valoarea unei funcții se apelează un subprogram funcție. Procesul computațional specificat de o funcție sau de o subrutină se numeşte **procedură**. Altfel spus, un subprogram extern defineşte o procedură. O procedură se poate apela din orice altă unitate de program.

Modulele şi blocurile de date nu sunt executabile; de aceea ele nu sunt considerate proceduri. O unitate de program BLOCK DATA conține numai definiții de date şi se foloseşte pentru a specifica valori inițiale pentru un anumit set de obiecte-date. **Modulul** conține definiții ce pot fi făcute accesibile altor unități de program. Modulul conține definiții de date, definiții de tip, definiții de procedură, numite **subprograme modul**, şi specificații de interfețe procedură. Subprogramele modul pot fi subprograme subrutină sau funcție. Un subprogram modul poate fi apelat de orice alt subprogram modul sau de orice altă unitate de program ce are acces la modul.

Programul principal, subprogramele externe şi subprogramele modul pot conține **subprograme interne** care pot fi subrutine sau funcții. Programul principal, sub-

programul extern sau subprogramul modul ce conține un subprogram intern se numeşte **gazda** (host) subprogramului intern. Subprogramele interne se introduc cu instrucțiunea CONTAINS. Subprogramele interne pot fi apelate de subprogramele lor gazdă sau de alte subprograme interne ale aceleiaşi gazde.

Toate unitățile de program, cu excepția blocurilor de date, pot conține blocuri de interfață. Un **bloc de interfață** descrie interfața la o procedură externă, adică numele procedurii, numărul de argumente, tipul lor, atribute, nume. Această informație permite procesorului să verifice dacă accesul a fost făcut în mod corect.

Structura unităților de program este arătată în Fig. 8.1.

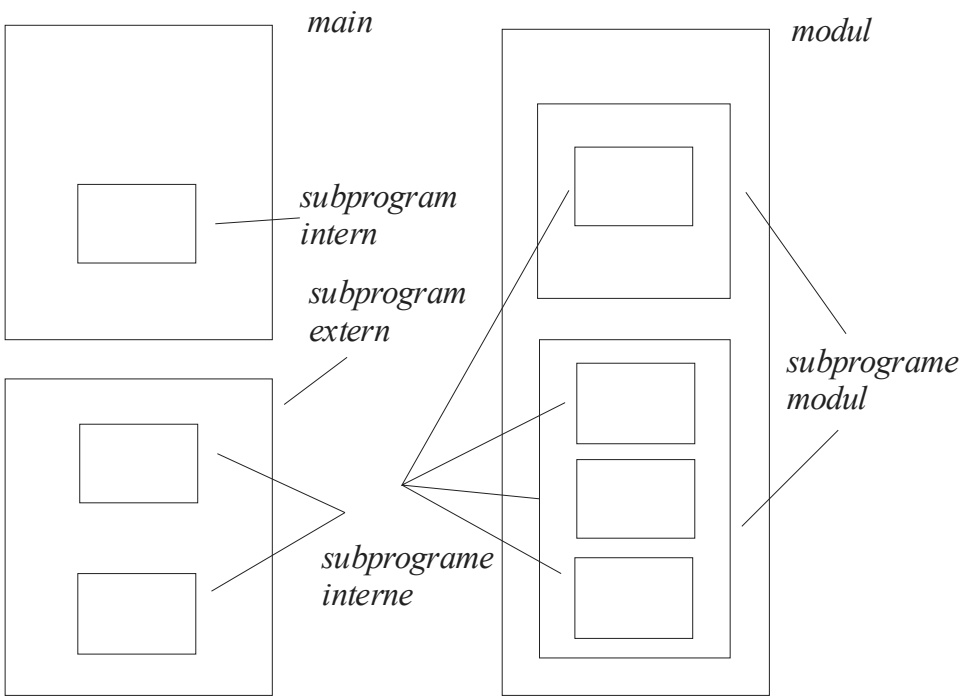


Fig. 8.1

8.2 Clasificarea instrucțiunilor Fortran

Instrucțiunile Fortran se pot împărți în instrucțiuni executabile şi instrucțiuni neexecutabile. O **instrucțiune executabilă** cauzează o acțiune, un proces de prelucrare a informației. O **instrucțiune neexecutabilă** nu provoacă acțiuni de procesare; instrucțiunea neexecutabilă este de natură informativă, ea descrie, clasifică sau specifică formatul unor elemente ale programului (date, unități de program, puncte de intrare). Instrucțiuni executabile sunt aşa-numitele **instrucțiuni de acțiune**. În cele ce urmează

descriem sintaxa instrucțiunilor de acțiune. Numele instrucțiunii este dat de un cuvânt cheie pe care îl scriem cu litere mari.

Formele instrucțiunilor de acțiune sunt:

ALLOCATE (*listă_de_alocație* [, STAT = *var_scal_int*])

ASSIGN *et* TO *var_scal_int*

BACKSPACE *unitate_de_fisier_extern*

CALL *nume_de_sub* [([*listă_de_spec_arg_act*])]

CLOSE (*listă_de_spec_close*)

CONTINUE

CYCLE [*nume_de_constr_DO*]

DEALLOCATE (*listă_de_nume* [, STAT = *var_scal_int*])

ENDFILE *unitate_de_fişier_extern*

ENDFILE (*listă_de_spec_de_poziție*)

EXIT [*nume_de_constr*_DO]

GO TO *et*

GO TO (*listă_de_et*) [,] *expr_scal_int*

GO TO *var_scal_int* [[,] (*listă_de_et*)]

IF (*expr_logică_scal*) *instr_de_acțiune*

IF (*expr_num_scal*) *et , et , et*

INQUIRE (*listă_de_spec_inquire*) [*listă_de_spec_output*]

NULLIFY (*listă_de_ob_pointer*)

OPEN (*listă_de_spec_de_conect*)

PAUSE (*cod_acces*)

PRINT *format* [, *listă_de_ent_output*]

READ (*listă_de_spec_de_control_io*) [*listă_de_ent_input*]

READ *format* [, *listă_de_ent_input*]

RETURN [*expr_scal_int*]

REWIND *unitate_de_fişier_extern*

REWIND (*listă_de_spec_de_poziţie*)

STOP [*cod_de_acces*]

WHERE (*expr_logică_tablou*) *instr_de_atrib_tablou*

WRITE (*lista_de_spec_de_control_io*) [*lista_de_ent_output*]

var_pointer => *expr_ţintă* (instrucţiune de atribuire pointer)

var = *expr* (instrucţiune de atribuire)

Mai sus am folosit prescurtările din tabelul 8.1.

Tabelul 8.1

Prescurtare	**Semnificaţie**	**Prescurtare**	**Semnificaţie**
act	actual(e)	*init*	iniţial(ă)
arg	argument(e)	*int*	întreg(întreagă)
atrib	atribuire	*instr*	instrucţiune
conect	conectare	*io*	input-output
constr	contrucţie	*num*	numeric(numerică)
dimens	dimensiune(i)	*ob*	obiect(e)
ent	entitate(entităţi)	*scal*	scalar(ă)
et	etichetă(e)	*sel*	selector
execut	executabil(ă)	*spec*	specificaţie(i)
expr	expresie	*sub*	subrutină
fin	final(ă)	*val*	valoare(valori)
incr	increment	*var*	variabilă

Prin **construcţie de control** înţelegem un anumit grup de instrucţiuni cu care se realizează instrucţiunile compuse ale programării structurate (§7.8), adică instrucţiuni executabile, de pildă instrucţiuni de acţiune şi instrucţiuni ce controlează ordinea în